

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 9

MODEL DEPLOYMENT — SPRINT 4

FastAPI, Streamlit, Public Hosting & Final Project Polish

The fourth and final sprint of the Phase 3 capstone: putting the model in front of real users. Interns serve their model with a FastAPI endpoint and a Streamlit dashboard, host it publicly, clean the repository to the Definition of Done, and close out the full four-sprint project.

PHASE 3 — SPRINT 4 • 40 HOURS • 5 TRAINING DAYS • DEPLOYMENT-FOCUSED

Prepared by BinX Tech
Palestine | Nablus

BinX

TECH

1. Week 9 Overview

Week	Week 9 of 10 — Phase 3: Deep Learning & Applied Project, Sprint 4 (final sprint)
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 17-18 combined)
Format	On-site / Remote / Hybrid — deployment-focused; code committed to GitHub via feature branches
Focus	Model deployment: FastAPI endpoints, Streamlit dashboards, public hosting (Hugging Face Spaces / Render / Railway), MLOps & reproducibility, repository polish; Sprint 4 of the Phase 3 capstone
Prerequisite	Weeks 1-8 completed; Sprint 3 reviewed; model integrated and fully evaluated
Mentor Supervision	Mentor Code & Notebook Review mid-sprint (Day 3) via pull request; Sprint Review + full-project Retrospective on Day 5

2. Sprint 4 Structure

Week 9 is Sprint 4 — the final sprint of the Phase 3 capstone project:

Event	Description
Sprint Planning (Day 1)	Define the Sprint 4 goal and backlog (serve, deploy, polish), incorporating the Sprint 3 retrospective improvement.
Daily Stand-up (Daily)	3-minute update: what was completed, what is next, any blockers.
Mentor Code & Notebook Review (Day 3)	Mentor reviews the deployment code and repository via GitHub pull request; structured comments before hosting.
Sprint Review (Day 5)	Demo the live, publicly deployed application. This closes the Phase 3 capstone project.
Sprint Retrospective (Day 5)	A full-project retrospective across all four sprints; preparation notes for the Week 10 final presentation.

3. Week 9 Learning Objectives

- Complete Sprint 4 planning and serialize the trained model for production use.
- Serve the model through a FastAPI REST endpoint with request/response validation.
- Build an interactive Streamlit dashboard that lets a non-technical user get predictions.
- Deploy the application publicly (Hugging Face Spaces / Render / Railway) at a live URL.
- Apply MLOps and reproducibility practices and polish the repository to the Definition of Done.
- Verify the project against the full Definition of Done and prepare for the Week 10 final presentation.

4. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Sprint 4 planning; model serialization; MLOps & reproducibility recap
Day 2	8 hrs	Serving the model with FastAPI: endpoints, Pydantic validation
Day 3	8 hrs	Building an interactive Streamlit dashboard; Mentor Code Review
Day 4	8 hrs	Public deployment (Hugging Face Spaces / Render / Railway); requirements & environments
Day 5	8 hrs	Repository polish, Definition of Done; Sprint Review & full-project Retrospective

5. Day-by-Day Curriculum

Day 1 — Sprint 4 Planning, Serialization & MLOps — 8 hours

Learning Objectives

- Complete Sprint 4 planning and define the deployment backlog.
- Serialize the trained model and its preprocessing objects for production.
- Apply reproducibility practices (pinned requirements, MLflow, seeds) ahead of deployment.

Key Topics

- Sprint 4 planning: goal, backlog, carrying forward the Sprint 3 retrospective
- Why deployment matters for a real portfolio
- Model serialization: joblib and Keras save
- Saving preprocessing objects to prevent skew
- MLOps recap: requirements.txt, MLflow, fixed seeds

Lesson Content

1.1 Sprint 4 Planning

Day 1 opens with Sprint Planning: define the Sprint 4 goal — deploy the model as a live, public application — break it into backlog tasks (serialize, serve, build UI, deploy, polish), and carry forward the Sprint 3 retrospective improvement. This sprint converts a trustworthy model into a usable product, satisfying the outline's requirement that every capstone reach a public URL.

1.2 Why Deployment Matters

A model in a notebook helps no one. Deployment is what turns a trained model into something real users can actually use — and it is the single feature that most separates a portfolio project employers take seriously from an academic exercise. The program's Definition of Done explicitly requires a live deployment at a public URL.

1.3 Model Serialization

Before serving a model, you must save it to disk so a separate application can load it without retraining. This is serialization. The method depends on the model type:

Model Type	Save Method	File
Scikit-learn / classical	joblib.dump()	model.joblib
TensorFlow/Keras	model.save()	model.keras or a SavedModel dir
Preprocessing objects	joblib.dump()	scaler.joblib, vectorizer.joblib

```
import joblib
joblib.dump(model, "model.joblib")
joblib.dump(scaler, "scaler.joblib")    # save preprocessing too!

# In the serving app, load them back:
model = joblib.load("model.joblib")
scaler = joblib.load("scaler.joblib")
```

Save the preprocessing objects (scaler, encoder, vectorizer) alongside the model. The serving app must apply the exact

same preprocessing as training — the training/serving skew rule from Week 8. Loading only the model and re-implementing preprocessing by hand is a top deployment bug.

1.4 MLOps & Reproducibility Recap

Deployment is where the reproducibility discipline built since Week 1 pays off. Three practices matter now: a pinned requirements.txt (or conda environment) so the deployment installs the exact library versions used in training; MLflow logs so every model version is traceable; and fixed random seeds so results are repeatable. Without these, a model that worked in the notebook may behave differently — or fail to install — in deployment.

Hands-On Lab: Sprint 4 Kickoff & Serialization

- Step 1: Complete Sprint 4 planning and select the deployment backlog tasks.
- Step 2: Serialize the trained model and every preprocessing object (scaler/encoder/vectorizer) to disk.
- Step 3: Write a small script that loads them back and reproduces a known prediction to confirm they work.
- Step 4: Freeze a clean, pinned requirements.txt for the deployment environment.

Tools Used

joblib / Keras save • MLflow • pip / requirements.txt • Jupyter/Colab • Git & GitHub

Day 2 — Serving the Model with FastAPI — 8 hours

Learning Objectives

- Build a FastAPI app that loads a serialized model and preprocessing.
- Create a /predict POST endpoint that validates input with Pydantic.
- Test the endpoint locally using FastAPI's automatic documentation.

Key Topics

- What FastAPI does: serving a model as a REST API
- Defining a POST prediction endpoint
- Pydantic input validation
- Applying saved preprocessing in the endpoint
- Testing via automatic /docs

Lesson Content

2.1 What FastAPI Does

FastAPI is a modern Python framework for building REST APIs — web endpoints that other programs can call. Serving a model with FastAPI means exposing a URL that accepts input data in a request and returns the model's prediction in the response. This is how models are served in production: the model becomes a service any application (a website, a mobile app, another backend) can query.

2.2 A Minimal Prediction Endpoint

A FastAPI app defines routes (URL paths) as functions. A POST endpoint receives input, runs the model, and returns the prediction as JSON:

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model = joblib.load("model.joblib")
```

```
scaler = joblib.load("scaler.joblib")

class InputData(BaseModel):    # defines & validates the expected input
    age: float
    income: float
    tenure: float

@app.post("/predict")
def predict(data: InputData):
    X = scaler.transform([[data.age, data.income, data.tenure]])
    pred = model.predict(X)
    return {"prediction": int(pred[0])}
```

2.3 Pydantic Validation

The InputData class above uses Pydantic, which automatically validates every incoming request against the declared types. If a caller sends text where a number is expected, or omits a field, FastAPI rejects the request with a clear error instead of crashing the model. This input validation is essential for a robust service — never trust that incoming data is well-formed.

2.4 Testing the API

FastAPI automatically generates interactive documentation at the /docs URL, where you can test every endpoint in the browser without writing a client. Running the server locally and calling /predict confirms the full chain works: request → validation → preprocessing → model → response.

```
# Run the server locally:
# uvicorn main:app --reload
# Then open http://127.0.0.1:8000/docs to test /predict interactively
```

The API applies the same preprocessing as training — loaded from the saved scaler, not re-implemented. This is the deployment payoff of saving preprocessing objects on Day 1.

Hands-On Lab: Building a Prediction API

- Step 1: Write a FastAPI app that loads the serialized model and preprocessing from Day 1.
- Step 2: Define an input schema with Pydantic matching the project's features.
- Step 3: Implement a /predict endpoint that preprocesses input and returns the prediction as JSON.
- Step 4: Run the server locally and test /predict through the /docs interface with several inputs.
- Step 5: Confirm invalid input is rejected cleanly by Pydantic, documented in the notebook/README.

Tools Used

FastAPI • Pydantic • Uvicorn • joblib

Day 3 — Interactive Streamlit Dashboard — 8 hours

Learning Objectives

- Build a Streamlit app that serves the model to non-technical users.
- Use appropriate widgets for the project's input type (numbers, text, or file upload).
- Design a clean, usable demo suitable for a live presentation.

Key Topics

- Why Streamlit: a UI for people, not programs
- Streamlit basics: widgets, script re-run model
- Common widgets by input type

- Designing a usable, focused demo
- Adding a visualization or explanation to the output

Lesson Content

3.1 Why Streamlit

A FastAPI endpoint is for programs; a Streamlit app is for people. Streamlit builds an interactive web app directly from a Python script — no HTML, CSS, or JavaScript required. It is the fastest way to give a non-technical user (a mentor, a recruiter, a stakeholder) a friendly interface to try the model, which is exactly what makes a project demo compelling.

3.2 Streamlit Basics

Streamlit turns simple Python calls into UI widgets. You read inputs from widgets, run the model, and display the result — the whole app is one top-to-bottom script that Streamlit re-runs on every interaction.

```
import streamlit as st
import joblib

model = joblib.load("model.joblib")
scaler = joblib.load("scaler.joblib")

st.title("Customer Churn Predictor")
age = st.number_input("Age", min_value=18, max_value=100)
income = st.number_input("Income")
tenure = st.number_input("Tenure (years)")

if st.button("Predict"):
    X = scaler.transform([[age, income, tenure]])
    pred = model.predict(X)[0]
    st.success(f"Prediction: {'Churn' if pred == 1 else 'Stay'}")
```

3.3 Common Streamlit Widgets

Widget	Purpose
st.number_input / st.slider	Numeric input
st.selectbox / st.radio	Choose from categories
st.text_input / st.text_area	Text input (e.g. for sentiment analysis)
st.file_uploader	Upload an image or CSV (e.g. for the image classifier)
st.write / st.success / st.plotly_chart	Display results, messages, and charts

3.4 Designing a Usable Demo

A strong Streamlit demo is more than a form: a clear title and short description, sensible input defaults and ranges, a prominent result, and — where it adds value — a visualization (a probability bar, a SHAP explanation from Week 8, or the uploaded image). Keep it clean and focused; the goal is for a first-time user to get a meaningful prediction in seconds. This is the interface demoed live in the Week 10 final presentation.

Hands-On Lab: Building the Demo + Mentor Review

- Step 1: Build a Streamlit app that loads the model and takes the project's inputs via appropriate widgets.
- Step 2: Run the model on user input and display the prediction prominently.
- Step 3: Add one supporting visualization (probability, SHAP explanation, or the uploaded image).

- Step 4: Run the app locally and confirm a first-time user can get a prediction easily.
- Step 5: Open a pull request with the deployment code for the mid-sprint Mentor Code Review and address the feedback.

Tools Used

Streamlit • joblib • Matplotlib / Plotly • Git & GitHub (pull request)

Day 4 — Public Deployment — 8 hours

Learning Objectives

- Deploy the application to a free public hosting platform.
- Configure requirements.txt and model files correctly for the build.
- Test the live deployment and reconcile any local-vs-deployed differences.

Key Topics

- Hosting options: Hugging Face Spaces, Render, Railway
- Deploying a Streamlit app to a public URL
- The critical role of a correct requirements.txt
- Handling model and preprocessing files in deployment
- Testing and debugging the live app

Lesson Content

4.1 From Local to Public

The app runs on your machine — now it must run on the internet at a URL anyone can open. Free hosting platforms make this straightforward for ML apps, each pulling your code from GitHub and building it automatically:

Platform	Best For
Hugging Face Spaces	Streamlit/Gradio ML demos — the simplest path, free GPU options
Render	FastAPI backends and web services, free tier
Railway	Full apps with automatic builds from GitHub

Hugging Face Spaces is usually the fastest route for a Streamlit demo: push the app and requirements.txt to a Space repository, and it builds and hosts automatically, giving a public URL in minutes.

4.2 The requirements.txt Is Critical

The hosting platform builds a fresh environment from your requirements.txt — if it is missing a library or has a wrong version, the deployment fails. This is where Day 1's pinned requirements pay off. Include every library the app imports (streamlit, scikit-learn, joblib, etc.) with the exact versions used locally.

```
# requirements.txt
streamlit==1.36.0
scikit-learn==1.5.0
joblib==1.4.2
numpy==1.26.4
pandas==2.2.2
```

4.3 Handling Model Files

The serialized model and preprocessing files must be included in the deployment repository (or downloaded at startup for large models). Verify the file paths in your code are relative and correct for the hosting environment —

a path that worked locally may not exist on the server. Large model files may need Git LFS or an external download step.

Deploy early and iterate: get a minimal version live first, then improve it. A live-but-simple app beats a polished-but-local one — the Definition of Done requires a working public URL, not perfection.

4.4 Testing the Live Deployment

Once deployed, test the public URL exactly as a real user would: submit several inputs, including edge cases, and confirm predictions match what the local app produced. A mismatch between local and deployed predictions almost always means a preprocessing or dependency difference — trace it before the Sprint Review.

Hands-On Lab: Going Live

- Step 1: Choose a hosting platform (Hugging Face Spaces recommended for Streamlit) and create the deployment repo/Space.
- Step 2: Push the app, the serialized model, preprocessing files, and a correct requirements.txt.
- Step 3: Confirm the build succeeds and obtain the public URL.
- Step 4: Test the live app with multiple inputs and confirm predictions match the local version.
- Step 5: Record the public URL in the project README.

Tools Used

Hugging Face Spaces / Render / Railway • GitHub • requirements.txt • Streamlit / FastAPI

Day 5 — Repository Polish, Definition of Done & Sprint Review — 8 hours

Learning Objectives

- Polish the repository and write the technical write-up to the Definition of Done.
- Verify the full project against every Definition of Done requirement.
- Complete the Sprint 4 review and a full-project retrospective across all four sprints.

Key Topics

- Polishing the repository: README, structure, artifacts
- The technical write-up: problem, method, results, limitations
- The full-project Definition of Done checklist
- Preparing for Week 10 (final presentation & certification)
- Sprint Review and full-project Retrospective

Lesson Content

5.1 Polishing the Repository

The GitHub repository is the artifact employers actually inspect, so it must be clean and complete. Per the program's Definition of Done, it needs: a clear README (problem statement, dataset, methodology, results, setup instructions, and the live URL), a pinned requirements.txt, the notebooks and scripts organized sensibly, model artifacts, and MLflow/experiment logs. No stray debug code, no unused files, no hard-coded local paths.

5.2 The Technical Write-Up

The project's short technical write-up (a required deliverable) states the problem, the dataset, the methodology, the results with metrics, and the limitations and future work. It is written for a technical reader and, together with the README and live demo, forms the core of the Week 10 presentation. Honesty about limitations is a mark of professional maturity, not weakness.

5.3 The Definition of Done — Full Project

The capstone project is complete only when every item is true, per the program's standard:

Requirement	Met When
Full pipeline	EDA → preprocessing → modelling → evaluation, implemented end-to-end
Benchmark met	The model hits a documented, reproducible performance benchmark
Deployed	A live Streamlit app or FastAPI endpoint at a public URL
Clean repo	README, requirements.txt, model artifacts, organized code
Experiments logged	All runs logged in MLflow or documented in the notebook
Reproducible notebook	Clean, commented, outputs present, fixed random seed

5.4 Sprint Review & Full-Project Retrospective

Sprint Review this week is also the close-out of the entire Phase 3 project: demo the live, public application to the mentor. The retrospective looks back across all four sprints — what went well, what would be done differently, and what to carry into the Week 10 final presentation. After this, the project is done; Week 10 is presentation, portfolio, and certification.

Hands-On Lab: Final Deployment & Project Close-Out

- Step 1: Write a complete README including the problem statement, methodology, results, setup steps, and the live URL.
- Step 2: Write the short technical write-up covering results, limitations, and future work.
- Step 3: Check the project against every Definition of Done item and resolve any gaps.
- Step 4: Ensure all Sprint 4 work is committed, the pull request is merged after mentor approval, and the repo is clean.
- Step 5: Present the Sprint Review (the live app) as the formal close of Phase 3, then write the full-project Retrospective for Week 10.

Tools Used

GitHub (README, repo cleanup) • Live deployment URL • MLflow • Technical write-up

6. Week 9 Deliverables

By the end of Week 9 (Sprint 4), every intern must submit the following to their mentor and GitHub repository:

- The Sprint 4 plan and the serialized model plus preprocessing objects, verified to reproduce a known prediction.
- A working FastAPI /predict endpoint with Pydantic validation, tested via /docs.
- An interactive Streamlit dashboard serving the model with appropriate widgets and a supporting visualization.
- The application deployed live at a public URL (Hugging Face Spaces / Render / Railway), matching local predictions.
- A polished repository (README with live URL, pinned requirements.txt, artifacts, logs) and a technical write-up.
- All Sprint 4 work committed via reviewed pull requests, plus the full-project Sprint Retrospective.

7. Week 9 Evaluation Criteria

Scored by the assigned mentor at the Sprint 4 review using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Deployment (Streamlit / FastAPI)	Partial or local-only deployment	Live public URL, functional	Polished UI, fast, handles edge cases
Serving correctness	Predictions differ local vs. deployed	Deployed predictions match local; validation works	Robust serving; no skew, clean error handling
Repository & reproducibility	Sporadic commits, incomplete README	Clean README, pinned requirements, artifacts present	Publication-quality repo; fully reproducible
Technical write-up	Missing or superficial	Clear problem, method, results, limitations	Insightful; honest limitations and future work
Sprint discipline & Git workflow	Misses steps, weak PR hygiene	Full sprint cycle, PR review, clear narrative	Proactive; strong full-project retrospective
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

8. Technical Stack — Week 9

Serialization	joblib, Keras model.save()
API Serving	FastAPI, Pydantic, Uvicorn
Interactive UI	Streamlit
Hosting	Hugging Face Spaces, Render, Railway
Reproducibility & Workflow	requirements.txt, MLflow, Git & GitHub (feature branches + pull requests)

9. Notes & Best Practices

Deployment is where reproducibility pays off: save preprocessing with the model, pin requirements.txt, and the deployed app will match the notebook. The recurring rule from Week 8 is decisive here — serving preprocessing must equal training preprocessing, or the live app quietly gives wrong answers. Deploy a minimal version early, then polish. After Sprint 4 the project is done and live; Week 10 presents it.